



WINDOW FUNCTIONS

Perform calculations (e.g. aggregation)
on a specific subset of data,
without losing the level of details of rows.

GROUP BY

Aggregates and groups rows based on column/s into summary row

ID	Product	Sales	
1	Caps	10	→
2	Caps	30	
3	Gloves	5	→
4	Gloves	20	

Product	Total Sales	
Caps	40	Σ
Gloves	25	Σ



WINDOW Functions

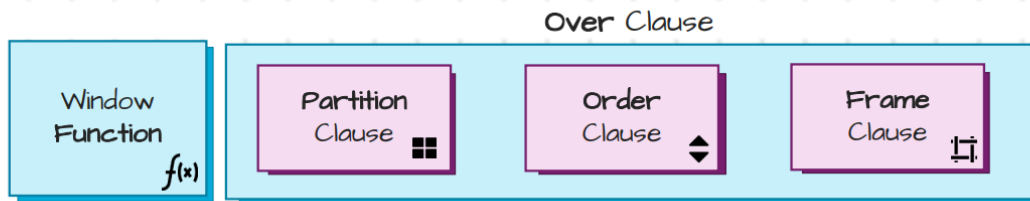
Compute **aggregates** but **keep details** of individual rows at the same time

ID	Product	Sales		ID	Product	Total Sales	
1	Caps	10	→	1	Caps	40	Σ
2	Caps	30	→	2	Caps	40	Σ
3	Gloves	5	→	3	Gloves	25	Σ
4	Gloves	20	→	4	Gloves	25	Σ

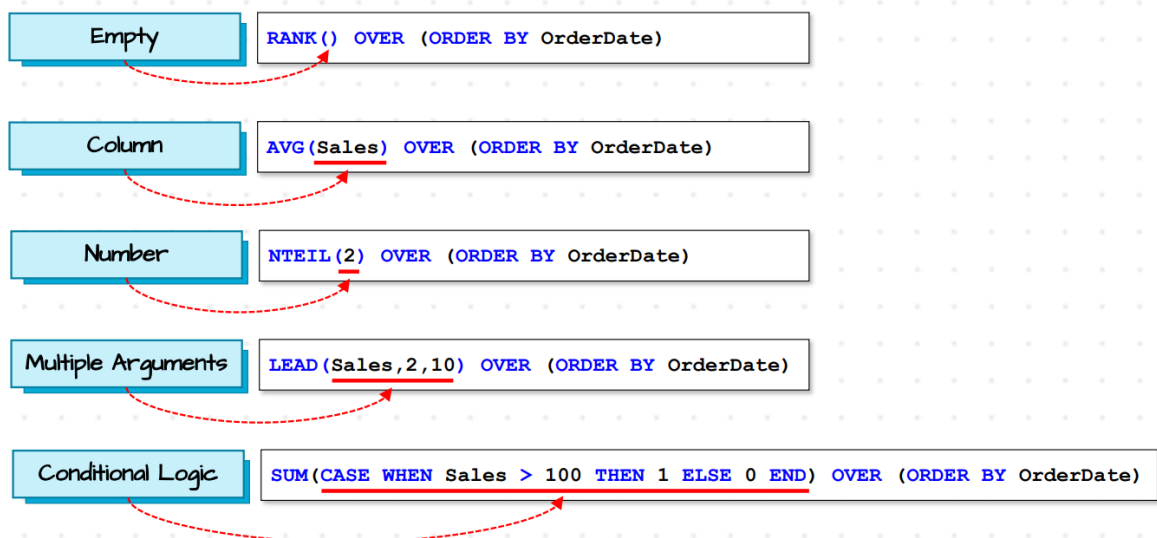
Row-Level Calculation

		Expression
Window Functions	Aggregate Functions	COUNT (expr) All Data Type
		SUM (expr) Numeric
		MAX (expr) Numeric
		MIN (expr) Numeric
		MIN (expr) Numeric
	Rank Functions	ROW_NUMBER () Empty
		RANK () Empty
		DENSE_RANK () Empty
		CUME_DIST () Empty
		PERCENT_RANK () Empty
		NTILE (n) Numeric
	Value (Analytics) Functions	LEAD (expr, offset, default) All Data Type
		LAG (expr, offset, default) All Data Type
		FIRST_VALUE (expr) All Data Type
		FIRST_VALUE (expr) All Data Type

Window Syntax

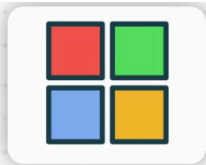


Window Expressions



OVER CLAUSE

Tells SQL that the function used is a window function



PARTITION BY

Divides the result set into partitions (Windows)

Course | Window Functions Basics | Partition By

Partition By



PARTITION BY divides the rows into groups, based on the column/s

Partition By



PARTITION BY divides the rows into groups, based on the column/s

Without
Partition By

Total sales across all rows (Entire Result Set)

```
SUM(Sales) OVER ()
```

Partition By
Single Column

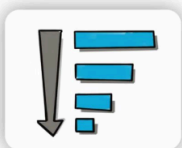
Total sales for each Product

```
SUM(Sales) OVER (PARTITION BY Product)
```

Partition By
Combined-Columns

Total sales for each combination of Product and Order Status

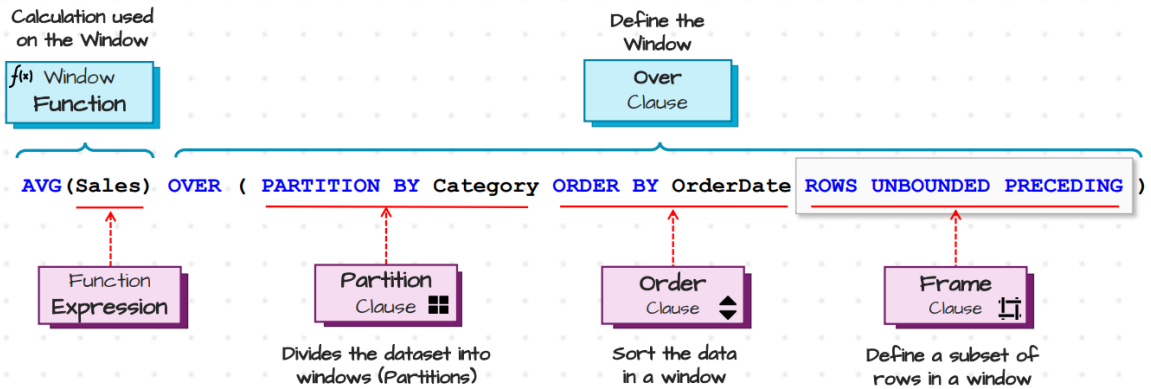
```
SUM(Sales) OVER (PARTITION BY Product, OrderStatus)
```



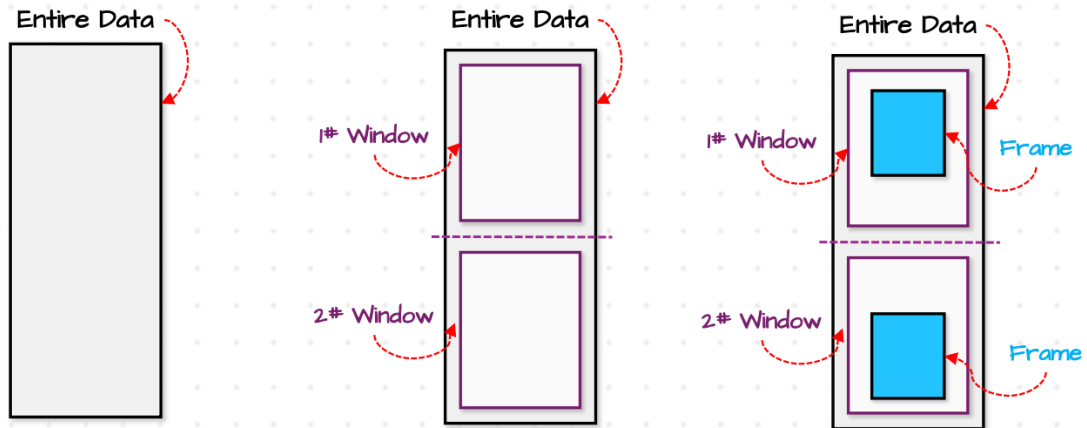
ORDER BY

Sort the data within a window

Window Syntax



Frame Clause

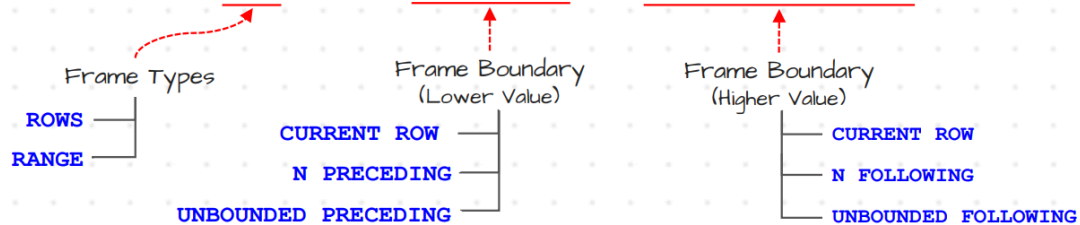


		Expression	Partition Clause	Order Clause	Frame Clause
Window Functions	Aggregate Functions	COUNT (expr) SUM (expr) AVG (expr) MAX (expr) MIN (expr)	All Data Type		
	Rank Functions	ROW_NUMBER () RANK () DENSE_RANK () CUME_DIST () PERCENT_RANK () NTILE (n)	Empty	Optional	Optional
	Value (Analytics) Functions	LEAD (expr,offset,default) LAG (expr,offset,default) FIRST_VALUE (expr) FIRST_VALUE (expr)	All Data Type	Optional	Optional

Frame

AVG(Sales) OVER (PARTITION BY Category ORDER BY OrderDate

ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)



Rules

- Frame Clause can only be used **together** with order by clause.
- Lower Value must be **BEFORE** the higher Value.

Window Rules



#1 RULE

Window functions can be used ONLY in **SELECT** and **ORDER BY** Clauses

#2 RULE

Nesting Window Functions is **not allowed** !

#3 RULE

SQL **execute** WINDOW Functions **after** WHERE Clause

#4 RULE

Window Function can be used together with GROUP BY in the same query, ONLY if the same **columns** are used

AVG Function



Default Average Function **exclude** NULLs



AVG(Sales) OVER(PARTITION BY Product)

Product	Sales	AVG
Caps	20	
Caps	10	
Caps	5	
$\frac{20 + 10 + 5}{3} = $		
Gloves	30	50
Gloves	70	50
Gloves	NULL	50
$\frac{30 + 70}{2} = 50$		

Deal with Nulls using **COALESCE** to **include** NULLs



AVG(COALESCE(Sales,0)) OVER(PARTITION BY Product)

Replace NULL with 0

Product	Sales	AVG
Caps	20	
Caps	10	
Caps	5	
$\frac{20 + 10 + 5}{3} = $		
Gloves	30	50
Gloves	70	50
Gloves	0	50
$\frac{30 + 70 + 0}{3} = 33$		

RUNNING TOTAL

Aggregate all values from the beginning up to the current point without dropping off older data.

ROLLING TOTAL

Aggregate all values within a fixed time window (e.g. 30 days).
As new data is added, the oldest data point will be dropped.

Rolling/Shifting Window

Overall Total

```
SUM(Sales) OVER()
```

Overview of entire data

Month	Sales	SUM
Caps	20	175
Caps	10	175
Caps	30	175
Gloves	5	175
Gloves	70	175
Gloves	40	175

Total Per Groups

```
SUM(Sales) OVER(  
PARTITION BY Product)
```

Compare Categories

Month	Sales	SUM
Caps	20	60
Caps	10	60
Caps	30	60
Gloves	5	105
Gloves	70	105
Gloves	40	105

Running Total

```
SUM(Sales) OVER(  
ORDER BY MONTH)
```

progress over time

Month	Sales	SUM
Jan	20	20
Feb	10	30
Mar	30	60
Apr	5	65
Jun	70	135
Jul	40	175

Rolling Total

```
SUM(Sales) OVER(  
ORDER BY MONTH  
ROWS 2 PRECEDING)
```

progress over time in
specific fixed window

Month	Sales	SUM
Jan	20	20
Feb	10	30
Mar	30	60
Apr	5	45
Jun	70	105
Jul	40	105

Ranking Function

	Expression	Partition Clause	Order Clause	Frame Clause
Rank Functions	ROW_NUMBER()	Optional	Required	Not allowed
	RANK()			
	DENSE_RANK()			
	CUME_DIST()			
	PERCENT_RANK()			
	NTILE(<i>n</i>)	Number		

Rank Functions	ROW_NUMBER()	Assign a unique number to each in a window	ROW_NUMBER() OVER (ORDER BY Sales)
	RANK()	Assign a rank to each row in a window, with gaps	RANK() OVER (ORDER BY Sales)
	DENSE_RANK()	Assign a rank to each row in a window, without gaps	DENSE_RANK() OVER (ORDER BY Sales)
	CUME_DIST()	calculates the cumulative distribution of a value within a set of values	CUME_DIST() OVER (ORDER BY Sales)
	PERCENT_RANK()	Returns the percentile ranking number of a row.	PERCENT_RANK() OVER (ORDER BY Sales)
	NTILE(n)	Divides the rows into a specified number of approximately equal groups	NTILE(2) OVER (ORDER BY Sales)



 DATA WITH F

ROW_NUMBER()

RANK()

DENSE_RANK()

ROW_NUMBER() OVER(ORDER BY Sales DESC)

RANK() OVER(ORDER BY Sales DESC)

DENSE_RANK() OVER(ORDER BY Sales DESC)

Sales	Rank
100	1
80	2
80	3
50	4
20	5

Sales	Rank
100	1
80	2
80	2
50	4
20	5

Sales	Rank
100	1
80	2
80	2
50	3
20	4

Unique Rank

Shared Rank

Shared Rank

Does NOT handle Ties

Handles Ties

Handles Ties

No Gaps in Ranks

Gaps in Ranks

No Gaps in Ranks

NTILE ()

Divides the rows into a specified number of approximately equal groups (Buckets)

DATA ANALYST

Data Segmentation

DATA ENGINEER

Equalizing
load processing

NTILE Use Case

NTILE (2) OVER (ORDER BY Sales DESC)

Sales	NTILE
100	1
80	1
80	1
50	2
30	2

$$\text{Bucket Size} = \frac{\text{Number of Rows}}{\text{Number of Buckets}}$$

$$2 = \frac{5}{2}$$

CUME_DIST()

Cumulative Distribution calculates the distribution of data points within a window

$$\frac{\text{Position Nr}}{\text{Number of Rows}}$$

PERCENT_RANK()

Calculates the relative position of each row

$$\frac{\text{Position Nr} - 1}{\text{Number of Rows} - 1}$$

CUME_DIST() OVER (ORDER BY Sales DESC)

PERCENT_RANK() OVER (ORDER BY Sales DESC)

Sales	DIST	Per
100	0,2	0
80	0,6	0,25
80	0,6	0,25
50	0,8	0,75
30	1	1

$$\text{CUME_DIST} = \frac{\text{Position Nr}}{\text{Number of Rows}}$$

$$\text{Percent_Rank} = \frac{\text{Position Nr} - 1}{\text{Number of Rows} - 1}$$

CUME_DIST



Calculates the **relative position** of a specified value in a group of values.

`CUME_DIST() OVER (ORDER BY Sales)`

Sales	Dist
20	0,2
50	
60	
80	
100	

$$\text{CUME_DIST}(x) = \frac{\text{Number of Rows less than or equal to } x}{\text{Total Number of Rows}}$$

$$\text{CUME_DIST}(20) = \frac{1}{5} = 0,2$$

PERCENT_RANK



Returns the **percentile ranking** number of a row

`PERCENT_RANK() OVER (ORDER BY Sales)`

Sales	Rank	Dist
20	1	0
50	2	0,25
60	3	0,5
80	4	
100	5	

$$\text{PERCENT_RANK}(x) = \frac{\text{Rank of } x - 1}{\text{Total Number of Rows} - 1}$$

$$\text{PERCENT_RANK}(60) = \frac{2}{4} = 0,5$$

Value Functions



Value (Analytics) Functions

LEAD (expr, offset, default)

Returns the value from a previous row

`LEAD(Sales, 2, 0) OVER (ORDER BY OrderDate)`

LAG (expr, offset, default)

Returns the value from a subsequent row

`LAG(Sales, 2, 0) OVER (ORDER BY OrderDate)`

FIRST_VALUE (expr)

Returns the first value in a window

`FIRST_VALUE(Sales) OVER (ORDER BY OrderDate)`

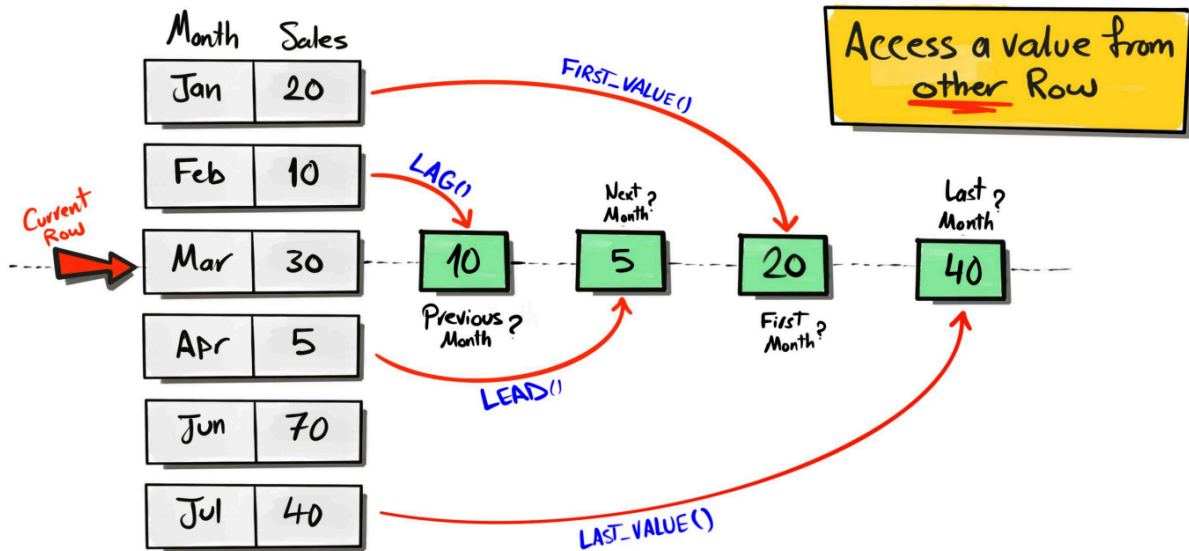
LAST_VALUE (expr)

Returns the last value in a window

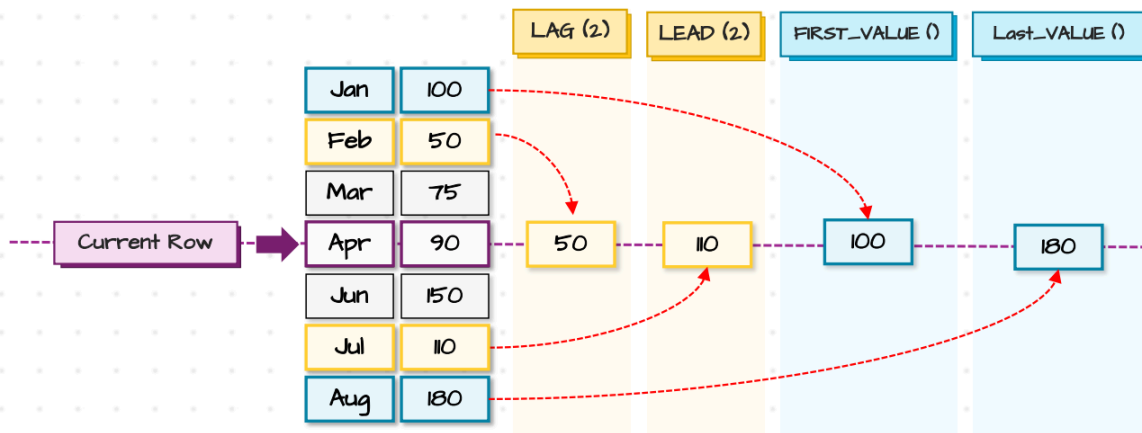
`LAST_VALUE(Sales) OVER (ORDER BY OrderDate)`

Value Functions

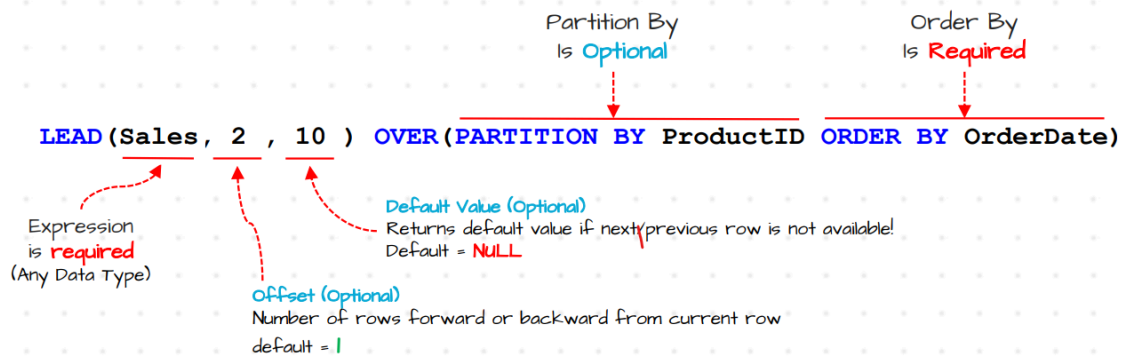
	Expression	Partition Clause	Order Clause	Frame Clause
Value (Analytics) Functions	<code>LEAD (expr,offset,default)</code>	All Data Type	Optional	Required
	<code>LAG (expr,offset,default)</code>			
	<code>FIRST_VALUE (expr)</code>			
	<code>LAST_VALUE (expr)</code>			
				Not allowed
				Optional
				Should be used



Value Functions



LEAD & LAG



LEAD & LAG



`LEAD(Sales) OVER (ORDER BY Month)`

Month	Sales	LEAD
Jan	20	10
Feb	10	30
Mar	30	5
Apr	5	NULL

Find Sales of the next month

`LAG(Sales) OVER (ORDER BY Month)`

Month	Sales	LAG
Jan	20	NULL
Feb	10	20
Mar	30	10
Apr	5	30

Find Sales of the previous month

FIRST & LAST



`FIRST_VALUE(Sales) OVER (ORDER BY Month)`

Month	Sales	First
Jan	20	20
Feb	10	20
Mar	30	20
Apr	5	20

`LAST_VALUE(Sales) OVER (ORDER BY Month)`

Month	Sales	Last
Jan	20	20
Feb	10	10
Mar	30	30
Apr	5	5

Default

`RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`

🧠 First: What are Window Functions?

👉 Simple definition:

A window function performs calculations across a set of rows **without collapsing them**.

Compare with **GROUP BY** (VERY IMPORTANT)

GROUP BY:

- Combines rows → returns fewer rows
- You lose original row-level detail

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department;
```

 Output:

- 1 row per department

Window Function:

- Keeps all rows
- Adds extra computed column

```
SELECT name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg
FROM employees;
```

 Output:

- Every employee still visible
- Plus department average column

Mental Model (this changes everything)

 **GROUP BY** = “collapse data”

 **WINDOW FUNCTION** = “annotate data”

Syntax Breakdown

```
<function>() OVER (  
  PARTITION BY ...  
  ORDER BY ...  
)
```

Components:

1. PARTITION BY

👉 Like **GROUP BY**, but **does NOT collapse rows**

- Splits data into groups

2. ORDER BY

👉 Defines order inside each group

3. (Optional) Frame

👉 Defines *range of rows* (advanced, we'll touch later)

Start with Basic Window Functions

1. Aggregate Window Functions

Same functions you know:

- **SUM()**
- **AVG()**
- **COUNT()**
- **MIN()**, **MAX()**

✓ Example

```
SELECT name, department, salary,  
       SUM(salary) OVER (PARTITION BY department) AS dept_total  
FROM employees;
```

👉 Each row shows:

- their salary
- total department salary

🟡 Pattern 1: Running Total

? Question:

"Find running total of salaries"

```
SELECT name, salary,  
       SUM(salary) OVER (ORDER BY salary) AS running_total  
FROM employees;
```

👉 Key:

- **ORDER BY** → defines accumulation order

🟢 Pattern 2: Row Number

? Question:

"Assign rank-like numbers"

```
SELECT name, department, salary,  
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS rn  
FROM employees;
```

👉 Gives:

- 1, 2, 3... inside each department

Pattern 3: Ranking Functions (VERY IMPORTANT)

1. ROW_NUMBER()

- Unique numbers
- No ties

2. RANK()

- Same rank for ties
- Skips numbers

3. DENSE_RANK()

- Same rank for ties
- No gaps

Example

```
SELECT name, salary,  
       RANK() OVER (ORDER BY salary DESC) AS rnk,  
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk  
FROM employees;
```

Difference:

Salary	RANK	DENSE_RANK
100	1	1
100	1	1

🟠 Pattern 4: Top N per Group (VERY COMMON INTERVIEW)

? Question:

"Get top 2 highest paid employees per department"

```
SELECT *  
FROM (  
    SELECT name, department, salary,  
           ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS  
rn  
    FROM employees  
) t  
WHERE rn <= 2;
```

👉 🔥 This replaces messy subqueries

🟠 Pattern 5: LAG / LEAD (VERY POWERFUL)

👉 LAG() → previous row

👉 LEAD() → next row

? Example:

"Compare salary with previous employee"

```
SELECT name, salary,  
       LAG(salary) OVER (ORDER BY salary) AS prev_salary  
FROM employees;
```

Output:

salary	prev_salary
30000	NULL
40000	30000

🔥 Real Use:

- trend analysis
- difference calculation

Example: Salary difference

```
SELECT name, salary,  
       salary - LAG(salary) OVER (ORDER BY salary) AS diff  
FROM employees;
```

● Pattern 6: FIRST_VALUE / LAST_VALUE

? Example:

"Get highest salary in each department"

```
SELECT name, department, salary,  
       FIRST_VALUE(salary) OVER (  
         PARTITION BY department  
         ORDER BY salary DESC  
       ) AS highest_salary  
FROM employees;
```

⚠ Common Confusion

❌ Why not use **GROUP BY**?

Because:

- You want **row + aggregate together**

🧠 When should your brain think “WINDOW FUNCTION”?

🔥 Trigger phrases:

Question says	Use
"for each row show..."	window function
"along with total/avg"	window
"rank / top N"	window
"running total"	window
"compare with previous row"	LAG
"keep all rows"	window

🚫 When NOT to use window

Situation	Use
Only aggregated result needed	GROUP BY
Filtering aggregated groups	HAVING

🧪 Full Example (combining everything)

```
SELECT name, department, salary,  
       AVG(salary) OVER (PARTITION BY department) AS dept_avg,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank,
```

salary - LAG(salary) OVER (PARTITION BY department ORDER BY salary) AS diff
FROM employees;



Final Intuition (SUPER IMPORTANT)

Before writing query, ask:

👉 "Do I want to **reduce rows** or **keep rows and add info?**"

- Reduce → **GROUP BY**
 - Keep + analyze → **WINDOW FUNCTION**
-